

Order Management Oracle Sql

let's walk through all the operations in your `Order_Pkg` stored procedures and functions. Your goal here is to build a modular and scalable backend for **order management**, designed around clean data flow, access control, and centralized logic.

Here's what each operation does, why it's needed, and how it contributes to the overall system



Create_Order



What it Does:

- Inserts customer **address details** into `Order_Address`
- Retrieves `address_id` from that insert
- Inserts an **order** into `Orders`, linking it with `address_id`



Why It's Needed:

- Keeps address and order insertion atomic and consistent
- Encapsulates logic so external callers don't have to manually resolve foreign key relationships
- Centralizes validation and error handling for clean API integration



Business Use Case:

When a customer places an order, both shipping info and order data are captured in a single call.



Update_Order_Status



What it Does:

- Updates the `status` field in `Orders` for a given `order_id`



Why It's Needed:

- Allows tracking of order progression (e.g., "Pending" → "Shipped")
- Ensures order lifecycle can be modified without exposing raw SQL externally



Business Use Case:

Used by fulfillment teams or systems to reflect order completion or cancellation.



Delete_Order



What it Does:

- Removes an order by its `order_id` from `Orders`
- Leaves `Order_Address` intact (in case multiple orders share the same address)



Why It's Needed:

- Supports cleanup or order cancellation scenarios
- Maintains referential integrity through managed deletion



Business Use Case:

Admin users can revoke incorrect or test orders while leaving addresses for reuse.



Get_Orders_By_Access_Level



What it Does:

- Accepts a company level (e.g., 'Local', 'Global')
- Determines which company names are visible based on hierarchy
- Returns all matching orders using a **pipelined function** (table-like output)



Why It's Needed:

- Implements **role-based access control**
- Allows dashboard views filtered by access tier
- Uses Oracle's pipelining for performance and flexibility in joins/filters



Business Use Case:

A regional manager can view orders from their level and below, while a global admin sees only global orders.



Get_Orders_By_Company



What it Does:

- Accepts a company name
- Returns orders for that company using a `SYS_REFCURSOR`
- Joins with `Company_Books` to show book titles

Why It's Needed:

- Targets a specific company for detailed reporting
- Uses ref cursor for flexible output in dynamic interfaces

Business Use Case:

This powers company-specific views, like "Show me all orders from Global and which books were ordered."

What You're Achieving

- **Separation of concerns** — business logic lives in the database, not in raw SQL in the UI or API layer
- **Security and consistency** — everything is validated and handled with error checks
- **Scalability** — access control, pipelined functions, modular design support real-world growth
- **Maintainability** — you can enhance logging, add audit trails, or refactor logic without touching outer systems